

Symbolic and Concolic Execution

1a)

```
1  int bounded_gcd(int a, int b) {
2      if (b != 0) {
3          int tmp1 = b;
4          b = a % b;
5          a = tmp1;
6
7          if (b != 0) {
8              int tmp2 = b;
9              b = a % b;
10             a = tmp2;
11         }
12     }
13     return a;
14 }
15
```

1b)

Let the initial symbolic state for the inputs be $a = \alpha$ and $b = \beta$. We must incorporate the precondition $\alpha > 0$ and $\beta > 0$ into our initial path constraint. Because $\beta > 0$, the first condition $b \neq 0$ is universally true, so execution always enters the first if block.

After the first unrolled iteration, the symbolic state is:

- $a \rightarrow \beta$
- $b \rightarrow \alpha \% \beta$

Now, the execution hits the second if ($b \neq 0$) condition, which branches based on whether $\alpha \% \beta = 0$. This yields two satisfiable paths:

- **Path 1 (True Branch):**
 - **Path Constraint:** $\alpha > 0$ and $\beta > 0$ and $(\alpha \% \beta \neq 0)$
 - **Final State:** $a \rightarrow \alpha \% \beta$, $b \rightarrow \beta \% (\alpha \% \beta)$
 - **Return Value:** $\alpha \% \beta$
- **Path 2 (False Branch):**
 - **Path Constraint:** $\alpha > 0$ and $\beta > 0$ and $(\alpha \% \beta = 0)$
 - **Final State:** $a \rightarrow \beta$, $b \rightarrow \alpha \% \beta$
 - **Return Value:** β

1.c)

To find positive values where the results differ, we need inputs that require more than two iterations of the Euclidean algorithm.

Let $a = 5$ and $b = 3$.

- `gcd(5, 3)`:
 - Iteration 1: $\text{tmp} = 3, b = 2, a = 3$
 - Iteration 2: $\text{tmp} = 2, b = 1, a = 2$
 - Iteration 3: $\text{tmp} = 1, b = 0, a = 1$
 - Result: 1
- `bounded_gcd(5, 3)`:
 - Iteration 1: $\text{tmp} = 3, b = 2, a = 3$
 - Iteration 2: $\text{tmp} = 2, b = 1, a = 2$
 - The function hits its bound and exits.
 - Result: 2

Meaning for Symbolic Execution: The unrolled version outputs incorrect results for paths that naturally exceed the bound. Is it an under-approximation? No. An under-approximation correctly computes a strict subset of the exact program behaviours. Because `bounded_gcd` returns false answers, it introduces incorrect behaviours do not present in the original `gcd`.

1.d)

To rule out executions not possible in `gcd` using path constraints, you can add an `assume(b == 0);` statement at the very end of `bounded_gcd`. This silently drops any symbolic execution path that would have required more than two iterations, ensuring that the engine only returns states that successfully and fully complete the GCD algorithm within the defined bounds.

2.a)

Inputs: $b = 0, e = 0$.

Execution Trace: `my_pow(0, 0)` initializes $r = 0$. The loop condition $1 < 0$ is false, so it returns 0. In `pow_client`, $0 \% 2 == 0$ is true, but $0 < 0$ is false, so the assertion is avoided.

Path Constraint Gathered: Let $b \rightarrow \beta$ and $e \rightarrow \epsilon$.

$\Phi = \neg (1 < \epsilon) \text{ and } (\epsilon \% 2 = 0) \text{ and } \neg (\beta < 0)$.

2.b)

Negating the Constraint We want to enter the if ($r < 0$) branch .

- Modified Path Constraint: We negate the last sub-constraint:

$\Phi_{\text{new}} = \neg (1 < \epsilon) \text{ and } (\epsilon \% 2 = 0) \text{ and } (\beta < 0)$.

- Satisfying Assignment: We need an even $\epsilon < 1$ and a $\beta < 0$. Let $e = 0$ and $b = -1$.

2.c)

- Inputs: $b = -1, e = 0$.
- Execution Trace: `my_pow(-1, 0)` sets $r = -1$. The loop is bypassed. It returns -1. In `pow_client`, $0 \% 2 == 0$ is true, and $-1 < 0$ is true.
- Path Constraint: $\neg (1 < \epsilon) \text{ and } (\epsilon \% 2 = 0) \text{ and } (\beta < 0)$.
- Result: Yes, we successfully reach the assertion, exposing the bug where b^0 incorrectly returns b instead of 1.

2.d)

Library Source Code Inaccessible If `my_pow` is a black-box library, the symbolic execution engine cannot track symbolic variables through it.

- First Run ($b=0, e=0$): `my_pow(0, 0)` executes concretely and returns 0.
- Because the engine couldn't trace the operations inside `my_pow`, the return value r concretizes to 0.
- Path Constraint Gathered: $(\epsilon \% 2 = 0)$. The condition $r < 0$ concretizes to $0 < 0$, so no symbolic constraint involving β is generated for the if statement.

2.e)

Finding Inputs Violating the Assertion You cannot proceed analogously. Because the relationship between the inputs (b, e) and the output (r) is lost when calling the uninstrumented library, the engine cannot mathematically deduce how to make $r < 0$. Concolic execution will likely never reach the assertion unless the random concrete inputs chosen for the initial runs happen to trigger it purely by luck.

2.f)

```
1 // my_pow.c
2 #include <stdio.h>
3 #include <assert.h>
4
5 int my_pow(int b, int e) {
6     int r = b;
7     for (int i = 1; i < e; i++) {
8         r = r * b;
9     }
10    return r;
11}
12
13 int main() {
14     int b, e;
15     // 1. Make inputs symbolic for KLEE to explore
16     klee_make_symbolic(&b, sizeof(b), "b");
17     klee_make_symbolic(&e, sizeof(e), "e");
18     // 2. Restrict 'e' to a small range to prevent infinite loops / state explosion
19     klee_assume(e >= 0 && e <= 2);
20     // 3. Define the expected mathematical behavior (Our Specification Oracle)
21     int expected;
22     if (e == 0) {
23         expected = 1;
24     } else if (e == 1) {
25         expected = b;
26     } else {
27         expected = b * b;
28     }
29     // 4. Assert that the implementation matches the specification
30     if (my_pow(b, e) != expected) {
31         klee_assert(false); // Expose the bug!
32     }
33     return 0;
34 }
```

```
robin.icy@robin:~$ klee -solver-backend=z3 -exit-on-error my_pow_test.bc
KLEE: output directory is "/home/robin.icyt/klee-out-0"
KLEE: Using Z3 solver backend
KLEE: Deterministic allocator: Using quarantine queue size 8
KLEE: Deterministic allocator: globals (start-address=0x7cd510800000 size=10 GiB)
KLEE: Deterministic allocator: constants (start-address=0x7cd298800000 size=10 GiB)
KLEE: Deterministic allocator: heap (start-address=0x7bd298800000 size=1024 GiB)
KLEE: Deterministic allocator: stack (start-address=0x7bb298800000 size=128 GiB)
KLEE: ERROR: my_pow_test.c:35: ASSERTION FAIL: false
KLEE: NOTE: now ignoring this error at this location
KLEE: ERROR: EXITING ON ERROR:
Error: ASSERTION FAIL: false
File: my_pow_test.c
Line: 35
assembly.ll line: 109
State: 1
Stack:
#000000109 in main() at my_pow_test.c:35
```

KLEE explores exactly 1 path before finding the error.

KLEE produces exactly 1 test case file named test000001.ktest in the klee-last/ directory.

When decoded using ktest-tool, the concrete inputs generated by the Z3 SMT solver for this path are:

- b (base): 0
- e (exponent): 0

The generated test case exposes the logic flaw in my_pow. The function assumes that initializing $r = b$ is safe for all exponents. However, mathematically, any number raised to the power of 0 (b^0) must equal 1. Because the function returns b when $e = 0$, it incorrectly evaluates $0^0 = 0$ or ($5^0 = 5$), which violates the mathematical specification.